NextRAN-AI

M1.2 The TensorPool Many-Core Cluster



Report by:
Marco Bertuletti, Ph.D. Student
Yichao Zhang, Ph.D. Student
yiczhang, mbertuletti @iis.ee.ethz.ch

Supervised by:
Prof. Alessandro Vanelli-Coralli
Prof. Luca Benini
avanelli, lbenini @iis.ee.ethz.ch

Integrated Systems Laboratory,
ETH Zürich,
Zürich, Switzerland

February 11, 2026

Contents

1 Introduction **2**

2 Architecture (WP1.Y1, WP1.Y2) **2**

2.1 TensorPool interconnect 3

2.2 AXI&DMA interconnect 5

2.3 Observations on the L1 memory balance 6

2.4 TensorPool's PE 8

2.5 TensorPool's TEs 8

2.6 Latency-tolerant Streamer 9

3 Software mapping (WP3.Y1) **10**

3.1 GEMM parallelization scheme 10

3.2 Parallelization scheme of PE operations 12

4 Assessment of TEs and PEs utilization (WP1.Y1, WP3.Y1, WP1.Y2) **12**

4.1 Results on single-TE GEMM and selection of hardware parameters 12

4.2 Results on parallel-GEMM 14

4.3 Results on PE operations 15

5 Physical Design (WP1.Y1, WP1.Y2) **17**

5.1 Floorplan guidelines 17

5.2 PPA analysis 18

5.3 SoA Comparison 20

6 Future work **20**

1 Introduction

In this Milestone we present the preliminary design of TensorPool, the basic computing block of NextRAN-AI computing platform. TensorPool integrates the RedMulE [1] Tensor Engine (TE) in TeraPool's L1 interconnect, and increase its peak-performance by 2.25× (8.3 TFLOPS@16b), to target General Matrix-Matrix Multiply (GEMM) dominated AI-native Radio Access Networks (RAN). Thanks to domain specialization we also obtain 6× better throughput on GEMM compared to TeraPool.

The content of this milestone addresses the tasks of WP1, and WP3 in the first project year, WP1 in the second project year. We defined the specification for the NextRAN-AI hardware platform, based on the computational requirements of existing models for AI-native Physical Layer (PHY) and on the benchmarking results for parallel key micro-kernels on TeraPool, our baseline hardware architecture. We finally develop improvements on the baseline architecture based on the requirements and the bottlenecks highlighted by the benchmarks. In particular:

1. In Section 2 we present Processing Elements (PEs), TEs and most importantly the compute to memory interconnect. We describe in detail the latency tolerance features that were added in the RedMulE to memory interface to tolerate the latency of TensorPool distributed L1 and operate the Floating-Point Multiply&Add Units (FMAs) at high utilization (up to 89%).

2. In Section 3 we describe the parallelization of GEMMs on TensorPool and we propose a solution to utilize all the compute resources of TensorPool in parallel, fully exploiting the flexibility offered by the programmable PEs, which can operate concurrently to the TEs.

3. In Section 5, we discuss the Power, Performance and Area (PPA) of TensorPool in TSMC's FinFet N7 technology node. We compare TensorPool with TeraPool and existing engines for AI-RAN and we demonstrate that it matches the requirements set in [2] as computing block for the NextRAN-AI platform.

4. In Section 6 we discuss the next steps of the project, including the placement and routing of the full TensorPool cluster and the evaluation of the system level in a higher abstraction simulator.

The RTL description of TensorPool architecture and the benchmarks described in this milestone are open-sourced [1].

2 Architecture (WP1.Y1, WP1.Y2)

In this section, we describe the architecture of TensorPool, an AI-enhanced flavour of the TeraPool [3] architectural template. TensorPool integrates 256 RV32IMAF cores for classical

[1] https://github.com/pulp-platform/mempool/tree/mbertuletti/redmule

signal processing and non matrix-multiply tasks, and 16 RedMulE TEs, to target at high speed and energy efficiency GEMM workloads common to both classical wireless signal processing, such as Beam Forming (BF), Minimum Mean Squared Error (MMSE), and Neural Network (NN) inference. In order, we describe the TeraPool-derived TensorPool interconnect, TensorPool processing elements, TensorPool's TEs and their connection to the cluster shared Tightly Coupled Data Memory (TCDM).

Offering up to 4096 tensor operations per cycle, 4MiB of shared memory for enhanced data-reuse, and 256 Instruction Set Architecture (ISA) programmable cores for higher flexibility, TensorPool architecture meets the requirements outlined in [2] and is suitable as the main computing block of the NextRAN-AI project.

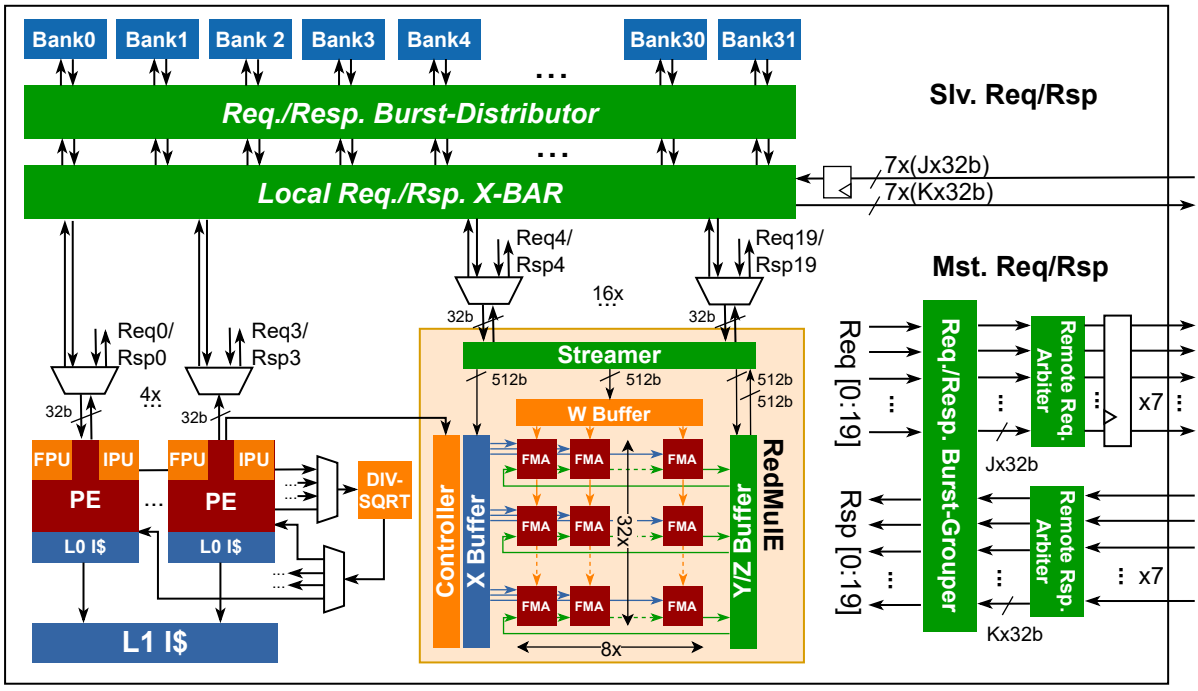


Figure 1: In TensorPool Tile the cores and the tensor unit connect to the local banks via a one cycle fully connected XBAR. The other banks of the distributed low-latenyc TensorPool L1 are accessed via the remote request and response arbiters. We represent the TE with H=8, L=32, P=3. The memory port is $16bit \times H \times (P+1) = 512bit$.

2.1 TensorPool interconnect

TensorPool is a TeraPool-based manycore cluster with 256 RV32IMAF PEs sharing 4 MiB of 2048-banked L1 scratchpad. Its *Tiles*, in Figure 1, contain 4 PEs, 32 2KiB memory banks, 4 KiB shared two-way set-associative L1-I$ and a 256 16-bit FMAs RedMulE engine. Memory within a Tile is accessed in 1 cycle via a fully-connected crossbar. Four Tiles form a *SubGroup*, and four SubGroups make a *Group*. The access to L1 memory in other Tiles goes through a remote-request arbiter and hierarchical crossbars, as in Figure 2: Tiles within a Subgroup communicate via an 4×4 crossbar; they access Tiles in other SubGroups of the same Group via

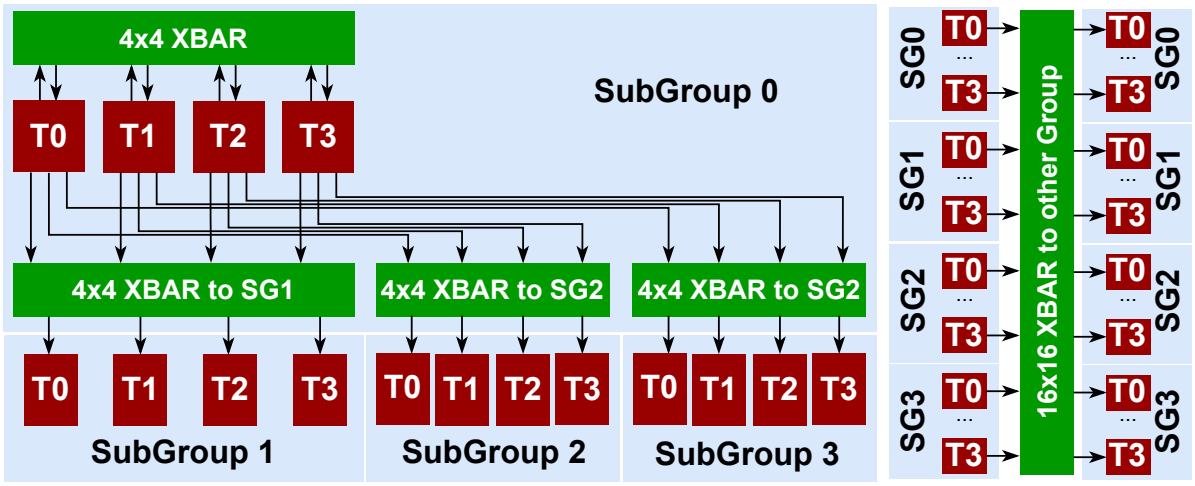


Figure 2: Connections between SubGroups in TensorPool. Tiles in a Subgroup connect to the portions of the scratchpad of Tiles in other SubGroups by 4x4 XBARs. The 16 Tiles of a Group connect to the portions of the scratchpad in Tiles of another Group via a 16x16 XBAR.

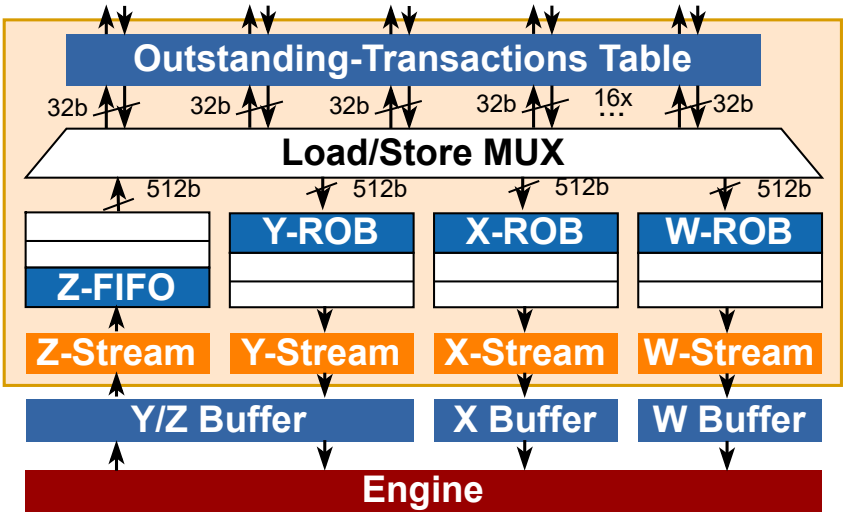


Figure 3: Streamer of RedMulE with the modifications included to improve the latency tolerance. We added three ROBs in the X, Y, W paths to pipeline multiple read requests in the interconnect and a FIFO on the Z path to hide stores while the accelerator loads new Y values in the Z/Y buffer.

additional three 4×4 crossbars. Tiles in a Group access the memory in Tiles in other Groups via three 16×16 crossbars. From the addition of spill-registers at the hierarchy boundaries to ease timing-closure, the access latency is non-uniform: 3 cycles for local SubGroup access, 5 cycles for local Group access, and 9 cycles for remote Groups without contentions in the shared interconnects. More in detail: a PE can access the memory banks within the Tile (64KiB) in 1-cycle, the memory banks within other Tiles of the same SubGroup (192KiB) in 3-cycles, the memory banks within Tiles of other SubGroups in the same Group (768KiB) in 5-cycles, and the memory banks within other Groups in 9-cycles (3MiB). The weighted mean of these delays gives the zero-load access latency: 7.84 cycles. Conflicts in the shared interconnection resources clearly increase on this baseline, as already discussed in related MemPool and TeraPool publications [3].

2.2 AXI&DMA interconnect

The TensorPool inherits its AXI-interconnect from TeraPool [3]. Each Tile has an Advanced eXtensible Interface (AXI) master port that is shared among all cores and the I$ refill. The AXI master grants access to higher-level/main memory (collecting all the off-L1 accesses), TensorPool cluster peripherals, and Control Status Registers (CSRs). However, routing a private AXI bus for each Tile to the cluster boundary is not physically feasible. To address this challenge, we have designed the AXI interconnect using a hierarchical approach. Each Tile acts as a leaf node in a configurable AXI tree. At each interconnect level, neighboring child nodes converge into a single AXI bus, resulting in each SubGroup having one 512 bit AXI master. Therefore TensorPool has 1024B/cycle memory bandwith shared between instructions and data. To efficiently move data to and from L1 memory, we utilize a Direct Memory Access (DMA) engine that leverages the AXI interconnect and accesses all 2048 Scratchpad Memory (SPM) banks in parallel. TensorPool has just one DMA engine, distributed in a unique frontend, a midend, acting as distributor, and multiple backends. The frontend, contaning the registers to program and initiate the transfer is placed in the cluster top level alongisde the peripherals and CSRs. Any PE in TensorPool can access the registers of the frontend to program a transfer. The PE's writes/reads to the DMA registers happen through the AXI interconnect. The midend splits the data transfer, sending start and stop addresses for different portions to the backends. Each SubGroup has its own backend. The backends receive the required information about the transfer from the midend and collect or send data to the AXI ports of 4 Tiles per SubGroup, to empty or refill memory banks.

As for the AXI tree this distributed DMA engine is a fundamental innovation, as it allows to distribute data to 2048 banks without the need of a frontend to all banks connection, which would be physically unfeasible. As a side note, we would like to clarify that besides keeping the same memory size and increasing peak compute capabilities of 2.25× (accounting the Te's only), compared to TeraPool, the TensorPool cluster is still balanced according to Kung's principle,

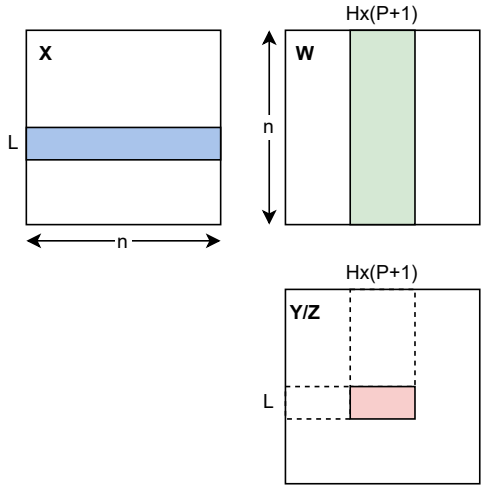


Figure 4: Tiling for the inner loop of the computation.

applied to a squared $n \times n \times n$ GEMM problem:

$$\begin{aligned}
&\pi_{TEs} = 16 \times 256\ MACs/cycle \text{ (peak compute)} \\
&\beta = 1024\ B/cycle \text{ (memory bandwidth)} \\
&W = n^3 MACs \text{ (MACs required by GEMM)} \\
&Q_m = 2B \times (n^2 + n^2 + 2 \times n^2) = 8n^2 B \text{ (transfers for a 16b double-buffered GEMM)} \\
&\left(T_{compute} = \frac{W}{\pi_{TEs}} = \frac{n^3 MACs}{8192\ MACs/cycle}\right) \geq \left(T_{transfer} = \frac{Q_m}{\beta} = \frac{8n^2\ B}{1024\ B/cycle}\right)
\end{aligned} \tag{1}$$

The double-buffering assumption requires half of the memory to be transferred: $Q_m = 8n^2B = 2MiB \rightarrow n = 512$. For this problem size Kung's disequation holds.

2.3 Observations on the L1 memory balance

We also want to demonstrate that the TE to L1-memory connection through the tightly-coupled interconnect is balanced according to Kung's principle. To demonstrate this result it is enough to demonstrate that the inner loop of the computation in Figure 4 is balanced. During the inner loop RedMulE computes an $H \times L \times (P+1)$ tile of the output matrix, it fetches L rows from X and $H \times (P+1)$ columns from W. Therefore:

$$\begin{aligned}
&W = L \times n \times H \times (P+1) MACs = 1024n\ MACs \\
&Q_m = 2B \times (L \times n + n \times H \times (P+1) + 2 \times L \times H \times (P+1)) = (128n + 2048)\ B
\end{aligned} \tag{2}$$

To simplify the following calculations, we can drop the Q_m term that does not depend on n. This means that the following considerations hold strong for a large enough problem over the dimension of the dot-products executed by the accelerator. Kung's principle can be simplified to this disequation:

$$\frac{W}{\pi_{TE}} \geq \frac{Q_m}{\beta} \rightarrow \frac{\pi_{TE}}{\beta} \leq \frac{W}{Q_m} \rightarrow \frac{\pi_{TE}}{\beta} \leq 8\ MACs/B \tag{3}$$

Let us first consider the memory within a Tile. It is easy to demonstrate that if the input matrices are only allocated in a Tile the connection is balanced:

$$\begin{aligned}
&\pi_{TE} = (H \times L)\ MACs/cycle = 256\ MACs/cycle \text{ (peak compute for a single TE)} \\
&\beta = H \times (P+1)\ B/cycle = 64\ B/cycle \text{ (memory bandwidth in the Tile)} \\
&\frac{256\ MACs/cycle}{64\ B/cycle} = 4\ MACs/B \leq 8\ MACs/B
\end{aligned} \tag{4}$$

Let's now consider remote memories. To further simplify the calculation we will assume that a TE does $H \times (P+1)$ wide access with a random starting address. In reality this is not the case, but the assumption works well when the matrices are big enough to be well distributed across all memory banks and the Xs and Ws do not overlap, which can be easily avoided using the tricks explained later in Section 3.

$$\begin{aligned}
&p_{IN} = 32\ Banks/2048 Banks = 0.0156 \quad \text{Probability of access within the Tile} \\
&p_{OUT} = (1 - p_{IN}) = 0.984 \quad \text{Probability of access outside the Tile} \\
&\beta_{local} = 64\ B/cycle \quad \text{Bandwidth of local (within a Tile) read ports} \\
&\beta_{remote} = 4\ B \times K/cycle \quad \text{Bandwidth of a remote (outside Tile) read port} \\
&\beta = p_{IN} \times \beta_{local} + p_{OUT} \times \beta_{remote} = 16.75\ B/cycle \\
&\frac{\pi_{TE}}{\beta} = 15.28\ MACs/B \leq 8\ MACs/B
\end{aligned} \tag{5}$$

With the grouping factor for remote read requests $K = 4$, the disequation seems to be violated. However, we considered only one read port active during the computation. Since the accelerator has support for 16 oustanding transactions over both the X and W streams we have to assume that it will be able to activate at least two ports. In fact, let us compute the probability that two independent remote accesses target the same port. It is a composite probability problem that can be easily solved:

$$\begin{aligned}
&N_{SG} = \text{Number of remote ports to other SubGroups} = 4 \\
&N_{G} = \text{Number of remote ports to other Groups} = 3 \\
&p_{G} = \text{Probability to hit the same Group than a previous request} = 1/4 \\
&p_{G\&SG} = \text{Probability to hit the same Group and also the same SubGroup} = 1/4 \times 1/4
\end{aligned} \tag{6}$$

Therefore, if the second request choses a remote Group port (3/4 cases) the probability of a double hit is p_G, while if the second request choses a port to another SubGroup (1/4 of the cases), the probability of a double hit is $p_{G\&SG}$. The overall probability of a double-hit is:

$$p_{double-hit} = 1/4 \times p_{G\&SG} + 3/4 \times p_G = 0.2 \tag{7}$$

Nonetheless, a double hit should occur for 16 independent accesses of X and 16 indepenedent accesses of W to only activate one port at a time, which means $\hat{p}_{double-hit} = (0.2)^{16}$. In conclusion,

we can compute a more sound estimate of the remote bandwith, which takes into account more remote ports active at a time during the computation. The Kung's disequation in this case is valid.

$$\begin{aligned} &\beta = p_{IN} \times \beta_{local} + p_{OUT} \times (p^{16}_{double-hit} + 2 \times (1 - p^{16}_{double-hit})) \times \beta_{remote} = 32.48\ B/cycle \\ &\frac{\pi_{TE}}{\beta} = 7.88\ MACs/B \leq 8\ MACs/B \end{aligned} \tag{8}$$

2.4 TensorPool's PE

The TensorPool PE, is based on the lightweight Snitch single-stage, single-issue core [4]. The core decodes and executes instructions in the RV32I base ISA in one cycle, it offloads multi-cycle instructions to specialized functional units: the Integer Processing Unit (IPU) and the Floating Point Unit (FPU). Snitch book-keeps the dependencies of offloaded instructions in a scoreboard and resolves them in program order. As a result, their additional latency is hidden as long as they do not have data dependencies. The IPU implements integer multiplication and division, and the Xpulpimg extensions, providing Single Instruction Multiple Data (SIMD) shuffling, packing, and unpacking of half types and are useful to manipulate complex 16 b data, typical in classical radio signal processing.

Snitch FPU supports Zfinx and Zhinx standard extensions, SIMD, widening dot-product and complex dot-product 16-bit floating point custom extensions. All floating-point instructions use only the 32 registers available in the RV32I core (also used for integer operations), eliminating the area overhead of floating-point Register File (RF) and dedicated floating-point load&stores. The FPU shares the same port of the IPU. The ISA is completed by floating-point division and square-root instructions, supported by a dedicated unit in TensorPool's Tile, which is shared between four cores.

2.5 TensorPool's TEs

RedMulE [1] is an inner-product based engine that accelerates $Z = Y \cdot (X \star W)$ operations between matrices tipycal of NN inference workloads, such as GEMM: $Z = Y + (X * W)$. RedMulE's FMAs are arranged in a grid with L rows and H columns, each FMA-row computes the dot-product between an X-row and a W-column, and sums the result to a Y-element. The computation is unrolled and the same X is reused for multiple W-columns, computed in parallel over the H engine columns. To fully utilize the compute resources, the engine must load $H \times$ 16-bit every cycle.

The FMAs can be pipelined. The option to add P pipeline stages ot the FMAs is a built-in latency tolerance feature of RedMule. In fact, the engine can load $(P + 1) \times H \times$ 16-bit every $(P + 1)$ cycles and preload the X and the Y values during the cycles in-between. This means that a W request has four cycles to be ready. The latency of memory operations is hidden by 3 data-buffers, each of size $H \times L \times (P + 1) \times$ 16-bit (the same buffer is used for Y and Z values), fed by the memory-streamer in Figure 3.

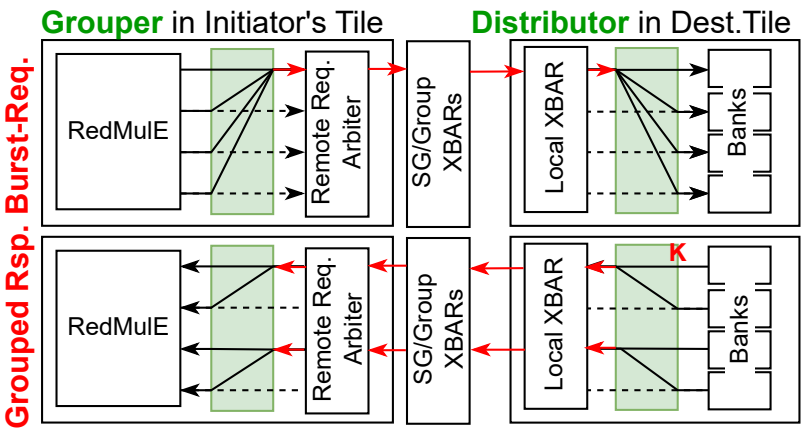


Figure 5: Representation of the Burst request and of the grouped response features. In the Burst Grouper of the initiating Tile multiple narrow read transactions to consecutive scratchpad addresses are combined in a single burst request (in red in the upper picture). In the Burst Distributor of the destination Tile the single burst request is redistributed to the target banks. For data responses, up to K responses can be grouped in a wider data response field (in red in the lower picture) and then redistributed to the RedMulE input ports in the initiating Tile. Similar approach can be also adopted for data-writes.

2.6 Latency-tolerant Streamer

The key challenge in TensorPool is to connect 16-TEs to all the distributed memory banks via a layered multi-cycle interconnect, while hiding memory latency and achieving full FMA-utilization. We achieve this goal by modifying the streamer to include a ROB on the X, W, Y, paths, enabling the issue of multiple outstanding memory reads to hide the latency of TensorPool interconnect. We also add a 32-entries FIFO-buffer on the Z path, to empty the Y/Z buffer upon the dot-product completion and pre-load new Y values without stalling on writes. A transaction table is used to collect the narrow 32-bit bank data-responses and commit $(P+1) \times H \times 16$-bit wide responses to the engine.

To avoid serialization of the wide requests at the arbiter that accesses TensorPool's distributed L1 memory system we adopt the burst transaction strategy proposed by [5]. On reads, only the first address of the wide request is sent by a *Burst-Grouper* in the Tile initiating the request. The burst read is then re-distributed to target banks by a *Burst-Distributor* in the destination Tile, as shown in Figure 5. Serialization of burst-responses at the local crossbar of the destination Tile can be partially avoided by grouping K 32b data-responses under the same handshake, effectively increasing the interconnect bandwidth. Figure 3 shows an example for $K = 2$. A similar approach can also be adopted for wide write-requests with a data-request widening factor J.

3 Software mapping (WP3.Y1)

In this section, we describe the mapping of GEMM workloads on TensorPool architecture. We explain in detail how one can exploit the two degrees of parallelism in TensorPool: parallelization on different TEs of independent GEMM workloads and parallelization of the same GEMM workload on all the TEs. We also describe the mapping of NN activations and classical signal-processing kernels on the PEs. We demonstrate how to build an L1 processing pipeline to accelerate Artificial Intelligence (AI)-native PHY workloads on both PEs and TEs.

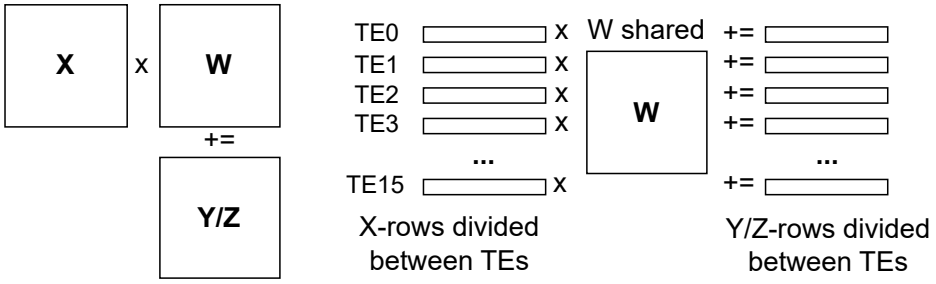


Figure 6: Parallelization of a GEMM on different TEs. The input X, Y and the output Z matrices are sliced along rows. Each slice is assigned to a different TE. The W matrix is shared.

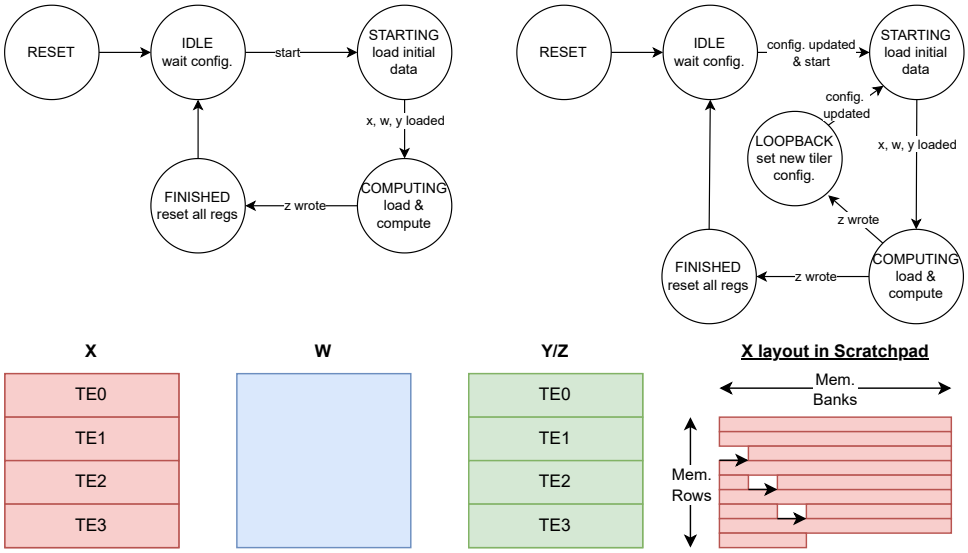


Figure 7: In the upper picture: diagram of the states for GEMM computation in a TensorPool TE and modified diagram, including the loopback state, where computation starts again from the first W column, in an execution where the offset was set. In the bottom picture: allocation of X in memory for the parallel execution of 4 TEs, to avoid banking conflicts.

3.1 GEMM parallelization scheme

TensorPool offers two main degrees of parallelism:

1. Medium to large size independent GEMM workloads can be parallelized over different

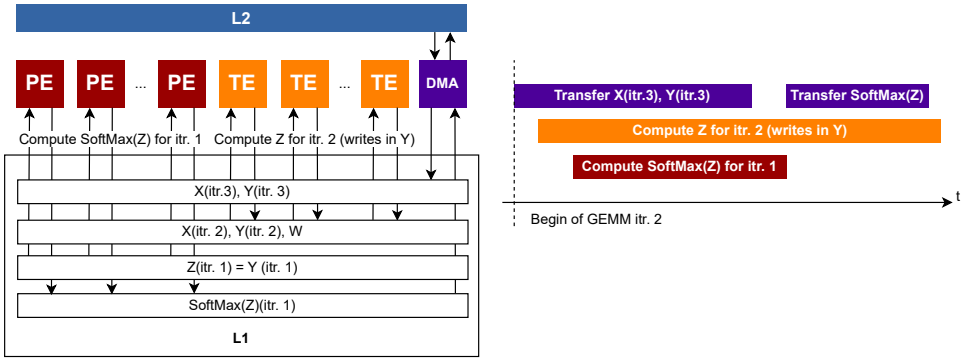


Figure 8: Example of L1 pipeline for concurrent PE and TE operation. While the PEs compute softmax on the output of GEMM, the TEs compute GEMM for next iteration and the DMA transfers out already computed data and brings in new inputs for the TEs.

TEs. Each TE access different X, W, Y, Z matrices. This feature can be patricularly useful during the multi-head attention computation, where one must compute small matrix multiplications, which are independent over the embedding or the time-domain dimensions.

2. Large to very large GEMM problems can be parallelized over the TEs. In this case, GEMMs are parallelized along rows of matrix X: each TE computes the dot-products between a subset of rows in X and all the columns of matrix W to produce a subset of output rows in matrix Z. The parallelization scheme is represented in Figure 6. This feature can be used to target beamforming.

In the second case, multiple TEs will access the same memory locations to fetch the same values in W, which is shared. Multiple TEs might also access the same banks when fetching values from different rows of X. To avoid these issues we adopt two strategies:

1. First, we want different TEs to start computing from different columns in W. Given a $M \times N \times P$ GEMM problem, where matrix X has size $M \times N$, matrix W has size $N \times P$, and matrices Y and Z have size $M \times P$, a TE can start computing from column p in W, compute the dot-products for the last $P-p$ columns and then loop back to compute the first $p-1$ columns. This approach avoids contentions in accessing the shared W matrix. The *loopback* feature can be supported in hardware, by adding an *offset* configuration parameter to RedMulE's configuration registers and by modifying the controller state-machine to empty the engine and start computing with a new startpoint on W, once the first portion of the output matrix, from column p, to column P is completed. The state machine for the TE execution including the *loopback* intermediate state is represented at the top of Figure 7.

2. Second, we can shift the starting point over the banks for the portions of matrix X assigned to different TE, as shown in Figure 7. With this solution, we can avoid that cores access the same banks while fetching from different rows of X.

3.2 Parallelization scheme of PE operations

We also tested the performance of PEs on activation and classical signal processing workloads. PEs represent an important addition to the TEs, as they allow to keep the flexibility required to target mixed classical signal processing and NN workloads. Moreover, they offer the possibility to pipeline GEMM with other operators, to complete the computation in L1 without moving the data to L2 and then back to the L1 of other computing elements with more general processing capabilities the TEs only.

Figure 8 represents an example of how to build this L1 pipeline. The output of a GEMM can be reused directly as input to the SoftMax computation. This allows us to hide the memory transfers from L2 over multiple iterations of GEMM and SoftMax (typical in NN-inference). The picture represents the second iteration of GEMM:

1. The TE are computing GEMM on data transferred from L2 in the first iteration. W is reused over iterations and can be kept in memory.

2. The PEs compute SoftMax on the output of the tensor unit for the first GEMM iteration. The PEs computation is completely overlapped with the TEs computation.

3. The DMA engine transfers from L2 the data for the third GEMM iteration and moves the softmax output from L1 to L2. The DMA transfers are completely overlapped with the TEs computation. DMA has to wait until the softmax computation is complete to transfer the data. However, if the GEMM calculation is long enough, this memory transfer is also hidden.

These operations can be executed without intermediate transfers between memory hierarchies and with simultaneous activity of TEs and PEs, aiming at full utilization of compute units.

4 Assessment of TEs and PEs utilization (WP1.Y1, WP3.Y1, WP1.Y2)

In this section, we benchmark different sizes of GEMM problems, and select the optimal architecture parameters (namely, the depth of ROBs and the width of the grouped request and response channels) to obtain a high TE-utilization. We also evaluate the impact of the proposed parallelization strategies on TEs utilization. We finally evaluate the runtime and utilization for activation functions and classical signal processing kernels on the platform.

4.1 Results on single-TE GEMM and selection of hardware parameters

In this sub-section, we present the benchmarking of different problem sizes on the platform to select the best sizing for the ROB depth and the request and response grouping factors K and J. We first present single-TE results.

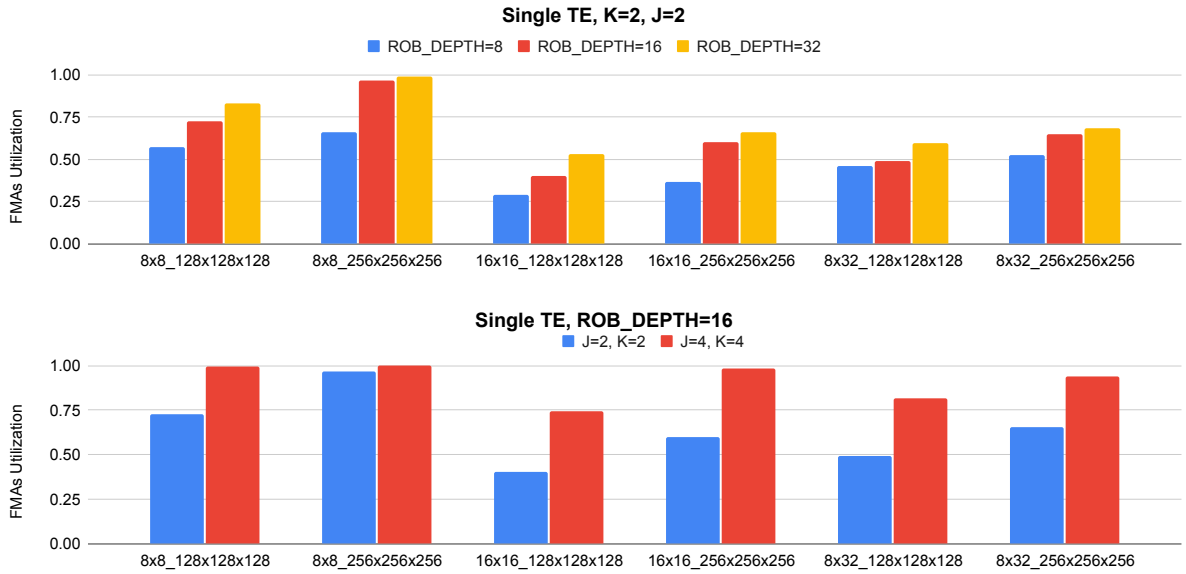


Figure 9: Results for GEMM executed on a single TE. On the horizontal axis we select different TE sizes (8x8, 16x16, 8x32) at different problem size (128x128x128 or 256x256x256). In the upper plot we keep the grouping factor on requests and responses fixed and we vary the ROB depth. In the bottom plot we keep the ROB depth fixed and we vary the grouping factor J, K.

The results in Figure 9 report the utilization of FMAs in two different experiments. In the upper plot, we fix the problem size and the size of the FMA array to either 8x8, 16x16 or 8x32 FMAs. We also fix the request and reponse grouping factors to J=2 and K=2. We report the different configurations that we choosed on the horizontal axis. We then vary the ROB depth parameter for each choosen configuration. With higher ROB depth, the TE can issue more transactions in the interconnect and retire them later in time, without blocking the streamer requests to memory. We notice that for large problem sizes the utilization obtained with a ROB depth of 16 is only only 0.02, 0.06 and 0.04 lower than the utilization obtained with a ROB depth of 32 in respectively the 8x8, 16x16 and 8x32 arrays. Therefore, to avoid doubling the ROB area and increasing the size in bits of the transaction ID pushed in the interconnect, we settle on a ROB depth of 16.

In the bottom plot, we fix the size of the TE and the size of the problem. We keep a ROB depth of 16 and we vary the grouping request and response factors J and K. A grouping response factor K=2 and K=4 indicates that the interconnect can carry, respectively, 64-bit and 128-bit data words in response to a read. The grouping of multiple response or request data under the same interconnection handshake is a simple way to reduce arbitration to the inputs of XBARs in TensorPool. However, increasing bandwidth through the grouping factor increases the complexity of the routing, as discussed in [5]. The choice on J and K is strongly dependent on routing resources available in the chosen technology for physical implementation.

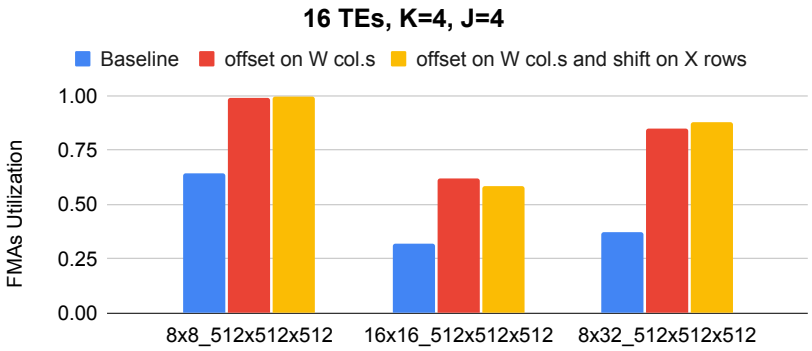


Figure 10: Results parallel GEMM. On the horizontal axis we select different TE sizes (8x8, 16x16, 8x32) on the same large 512x512x512 problem size. We consider different approaches on the application mapping and allocation of the inputs in L1.

For a fixed ROB depth and for fixed J and K, for example the selected 16 entries, J=4 and K=4, we explain the dependency on the utilization by the size of the inner-product TE and by the problem size in three points:

1. First, a RedMulE engine of size $H \times L$ executes on each row of the TE array the dot-products between one row of the X matrix and H×((P+1) columns of W, to compute H×(P+1) elements in Z. Therefore an $H \times (P+1) \times L$ tile in Z is processed in parallel on the TE elements. The larger is the matrix problem $M \times N \times P$ along the N dimension, the lower is the overhead of loading Ys and storing Zs on the TE operation. Therefore, the engine works better on large N.

2. Additionally, the same X value is reused for H values in W, therefore the larger H, the lower the overhead in fetching X values. This can be seen in the comparison between the $256 \times 256 \times 256$ GEMM run on the 16×16 array and the $256 \times 256 \times 256$ GEMM run on the 8×32 array.

3. Nevertheless increasing H can also requires a larger port of the TE to memory: $H \times (P+1)$. As a result the small H engines such as the 8×8 one can always saturate utilization on large problem size, because the latency of its 256b accesses can be hidden, while the 16×16 and 8×32 struggle, especially on smaller problem sizes, where the overhead of fetching Y and storing Z is big, as discussed. This effect is even more pronounced during parallel operation, because to the latency for arbitration we sum the latency cause by multiple TEs accessing the same banks (bank conflicts).

4.2 Results on parallel-GEMM

In this sub-section, we present the utilization of FMAs for different parallel GEMM sizes. We show that the two solutions devised in Section 3.1 to avoid contentions when accessing W and X are effective.

The results are reported in Figure 10. Shifting the starting point of the calculation for different TEs columns in the W matrix reduces the contentions while accessing the weights, improving the FMAs utilization, from 0.37 to 0.85. Deciding on an optimal allocation for the X matrix in memory can also be beneficial. The measured improvement depends on the size of the problem and the size of the array. In the case of an 8x32 array and a 512x512x512 problem, we further increase the utilization from 0.85 to 0.88.

From our previous experience, based on the work in [5], we select J=2 and K=4 for the backend trials. Additionally, based on the requirements of the projects and on the chosen interconnect configuration, we must select a TE size that can provide up to 256 MACs per cycle. The H=8 and L=32 RedMulE configuration requires a lower bandwidth than the H=16 and L=16 configuration, providing the same number of MACs per cycle. Therefore, we settle on an 8x32 RedMulE size with a 16 entries ROB, J=2 and K=4. In Figure 11 we report the runtime and utilization for different sizes of GEMM in the selected TensorPool configuration.

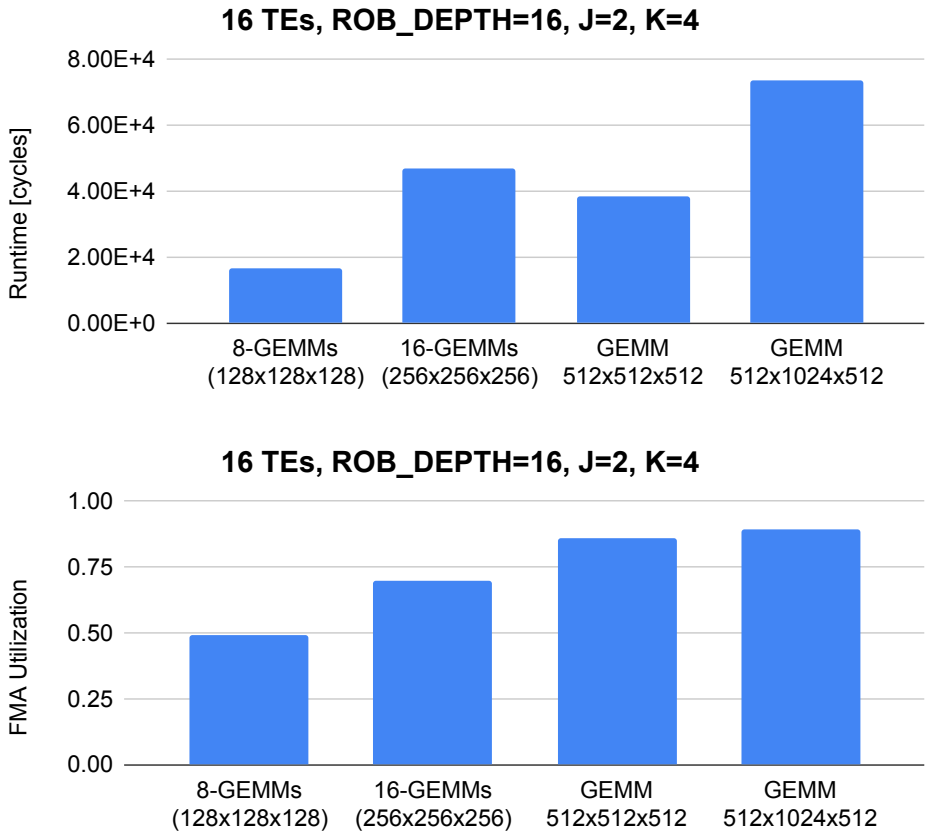


Figure 11: Runtime and FMA utilization for different GEMM sizes in the selected TensorPool configuration.

4.3 Results on PE operations

In Figure 12 we report the runtime of activation functions with the same input dimension of the matrix multiplication problems addressed in Figure 10. We notice that parallel Batchnorm,

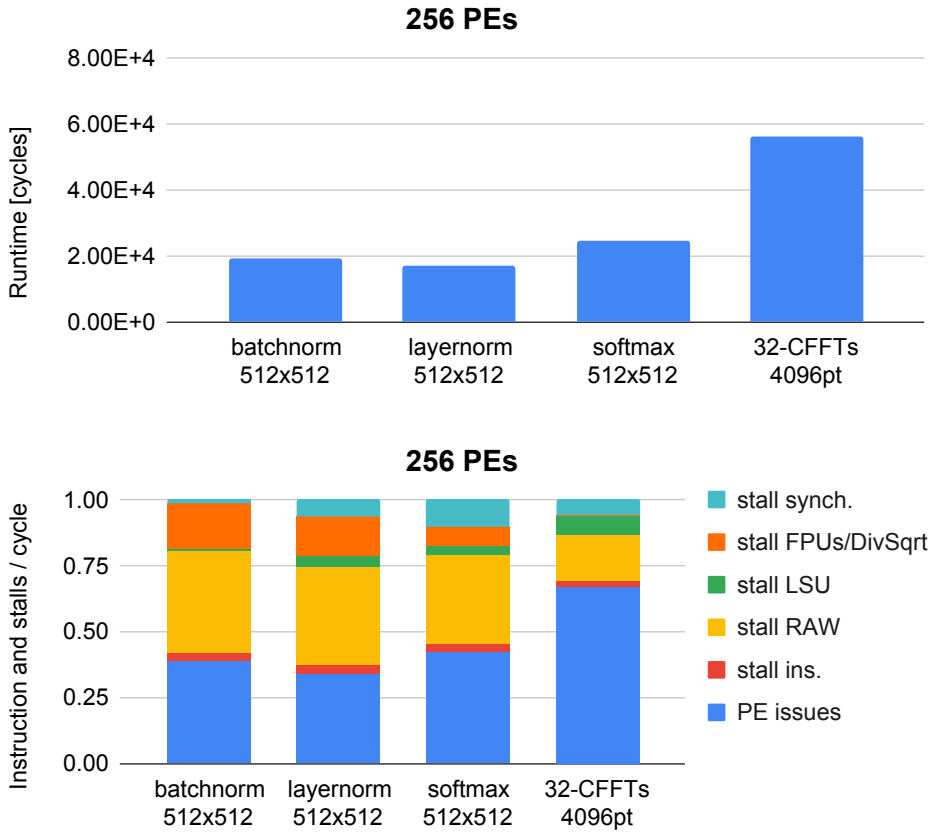


Figure 12: Runtime, breakdown of instructions and stalls for deep learning activation functions and FFT, executed in parallel on the PEs.

Layernorm, and Softmax have smaller runtime than an equal-size GEMM, enabling pipelined overlapped execution of dense-layers and activations in a NN workload, as described in Figure 8.

We also report the breakdown of the instructions and stalls over the total cycles, for the PEs computation. The instructions-per-cycle (IPC) refers to the fraction between instruction issues and cycles runtime. The synchronization stalls refer to the time spent by PEs waiting for the completion of the parallel task by other PEs. The read-after-write (RAW) stalls are caused by the functional units not ready to provide the compute data to a new instruction, while the functional units (FPUs and DivSqrt stalls) refer to stalls of the units themselves (the pipeline is full and cannot accept new operands). Instruction stalls are caused by misses in the shared I$ and Load Store Unit (LSU) stalls refer to the LSU's outstanding transactions table being full and unable to accept new reads or writes. The main source of stalls in the execution of the activation is the stalls of functional units. In particular, the stalls of the shared division and square root unit, which is used intensively by the four cores of a Tile during parallel execution, are the main contribution.

We also report the breakdown of the instructions and stalls over the total cycles for the execution of $32 \times 4096pt$ FFTs. The IPC (0.66) and the runtime are aligned with those measured in [6], remarking on the flexibility of our architecture.

In Figure 13 we report the runtime for the application described in Figure 8. To simulate the main memory latency with TensorPool we utilize the open-source DRAMSys5.0 simulator [7], a cycle-accurate framework based on SystemC TLM-2.0 for Dynamic Random-Access Memory (DRAM) subsystems. We configure main memory with two DRAM stacks, each with 8 Micron MT54A16G808A00AC-36 HBM2E channels, which support 2.8/3.2/3.6 Gbit/s/pin Double Data Rate (DDR) transmission. Overlapping the execution of DMA transfers, GEMM and softmax, fully exploting the parallelism offered by the TensorPool architecture we can reduce the runtime of 25% compared to a serialized execution. The experiment demonstrates that the overlapped execution is not bottlenecked by main memory transfers. However, we notice a lower utilization of the TEs FMAs (0.58 vs 0.85): during parallel execution, DMA and PEs access the same banks targeted by the TEs, causing banking conflicts.

In our future work we will focus on the proposed L1-Transformer application. We will evaluate the utilization of the PEs and TEs on the most computationally intensive blocks. It was previously demonstrated that the DMA architecture occupies less than 5% of the total cluster area and burns less than 5% of the total power [3]. Moreover, we demonstrated that the architecture is balanced with respect to L2 under the provided DMA bandwidth. It is therefore not necessary to seek PPAs optimizations on the DMA and AXI-interconnect side.

Once running the full application, our focus will be on the utilization of the PEs and the TEs. In particular, we agree that in case the concurrent PEs and TEs execution represents the dominant pattern, the number of PEs should be chosen to have the same runtime between the tensor execution and the scalar cores execution, so as to optimize PPAs. In case significant blocks of the code require only PEs processing (e.g. permutations, image to column transformations, activations, etc.), the number of PEs should be calibrated to avoid excessive runtime on those blocks and keep real-time operation of the model.

5 Physical Design (WP1.Y1, WP1.Y2)

In this section, we describe the physical design of TensorPool in TSMC-N7. We provide guidelines for the physical implementation of our design and we describe the PPA results and how they compare to the requirements set in [2]. We also compare our work with state of the art processors for AI-native RAN.

In this document, we report the preliminary results for a placed and routed instance of TensorPool SubGroup in TSMC N7. We use Synopsys Fusion Compiler 2025.06 for placement and routing and Synopsys Primetime 2022.03 for power simulations. The work on placement and routing of the full TensorPool cluster is ongoing.

5.1 Floorplan guidelines

TensorPool has the same design hierarchy as TeraPool (Tile, SubGroup, Group, Cluster). Its physical design respects the hierarchical bottom-up approach adotped for TeraPool too. Tiles

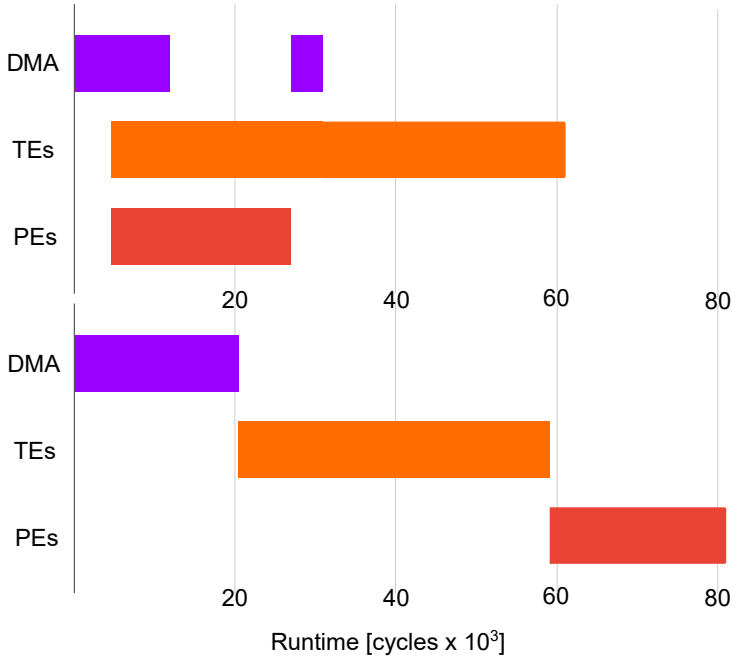


Figure 13: Runtime of the GEMM and softmax pipeline for a 512x512x512 problem size. The GEMM is executed on the TEs, while the softmax is executed on the PEs. DMA takes care of the memory transfers. On the top, the DMA, TEs', and PEs' operations are overlapped. On the bottom, they are serialized.

are flattened in a SubGroup. The placed and routed SubGroup instances are then assembled in a Group, where we also place and route the Group to Group XBARs. The cluster flow only requires routing the point to point connection between SubGroups.

When specifying the timing constraints for the SubGroup and Group input and outputs we leave enough margin to ensure timing closure in the upper hierarchical level of the bottom-up flow, respectively the Group and the Cluster.

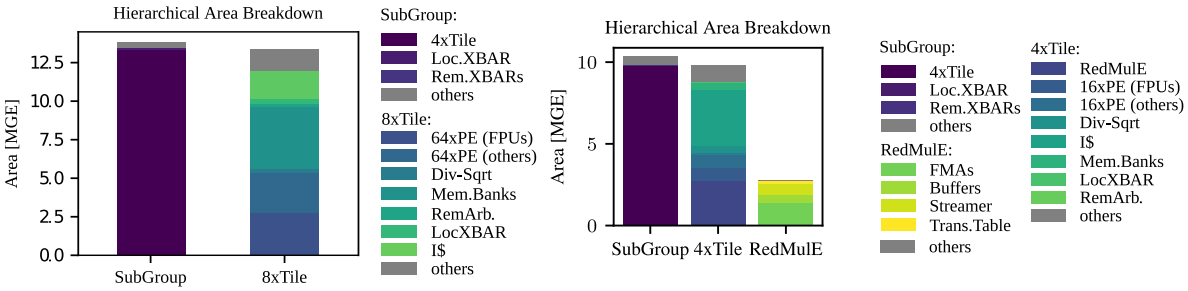


Figure 14: Breakdown of the SubGroup cell area in the TeraPool and TensorPool designs.

5.2 PPA analysis

An instance of TeraPool Subgroup in TSMC N7 technology placed with Synopsys Fusion Compiler 2025.06 and with frequency target 800MHz closes timing in the TT, 25°C, 0.75V corner at 900MHz. The longest path is within the single-stage pipeline of the core, and the result is

Table 1: TensorPool improvement over TeraPool

	TeraPool	TensorPool
Node	12nm	7nm
Area (SubGroup) [mm^2]	3.03	0.91
Frequency (TT-25°C) [GHz]	0.88	0.9
Peak-Performance (TEs) [TFLOPS@16b]	0	7.4
Peak-Performance (TEs + PEs) [TFLOPS@16b]	3.6	8.4
GEMM - Throughput [MACs/cycle (FP16)]	609	3643 6×
GEMM - Performance [TFLOPS@16b]	*1.1	6.6 6×
GEMM - Power (TT-25°C,0.75V) [W]	*6.3	4.3
GEMM - Area Efficiency [TFLOPS@16b/mm^2]	*0.07	0.46 6.8×
GEMM - Energy Efficiency [TFLOPS@16b/W]	*0.17	1.53 8.8×
GEMM - Energy&Area Efficiency† [GFLOPS@16b/W/mm^2]	*11	106 9.9×

*The critical path of TensorPool is within a PE, we compare the two designs at the same target frequency.

*Power & area technology normalized, multiplying by $(\frac{0.75V}{0.8V})^2, (\frac{7}{12})^2$.

†Based on SubGroup area.

aligned with other implementations of the Snitch core in the same technology.

Figure 14 is a breakdown of the SubGroup cell area for the TeraPool and TensorPool designs. A TeraPool SubGroup contains 8 Tiles, 8 PE per Tile, and no TE. It delivers 128 MACs/cycle in 16bit floating-point precision. A TensorPool SubGroup contains 4 Tiles, 4 PEs per Tile, and a RedMulE engine. It delivers 288 MACs/cycle in 16bit floating-point precision.

In TeraPool the compute area (area of the FPUs) corresponds to roughly 2.8 MGE. In TensorPool's SubGroup, the compute area (FPUs and FMAs) corresponds to 1.9MGE. Therefore, TensorPool delivers 2.25× the maximum throughput with 1.5× less area dedicated to the compute elements. This is because the FMAs of a TE only execute floating-point multiplication and addition operations. In contrast, the FPU in a PE supports the full set of *zfinx* and *zhinx* instructions. It additionally supports custom extensions to maximize the numerical precision on traditional signal processing workloads. Moreover, the FPU requires logic for instruction decoding. All these features are necessary to make TeraPool PEs flexible and to increase the range of workloads supported by the architecture. However, they reduce area efficiency, and they can be dropped in favor of a specialized accelerator, as in TensorPool, when addressing less heterogeneous workloads. Such as AI-native RAN, dominated by GEMM. TensorPool privileges GEMM workloads, targeting domain specialization. The number of programmable PEs in a SubGroup is reduced from 64 (5.34 MGE, 38% of the total SubGroup area) to 16 (1.3MGE, 13% of the total SubGroup area). This also has a beneficial effect on the size of the instruction cache, which occupies 1.8 MGE in TeraPool and 0.4 MGE in TensorPool.

In TensorPool, RedMulE occupies 25% of the SubGroup area. To hide the memory latency, we include ROBs and a FIFO. This adds to the total area of the accelerator. In fact, it represents the most relevant contribution: 49% of RedMulE's area. A key direction to explore in order to improve the accelerator area efficiency is the reduction of buffers. In particular, we are exploring

outer product engines. The aim is to reduce the size of the accelerator buffers by exploiting the FMA pipeline registers for double-buffering.

The results for GEMM execution on TeraPool and TensorPool are reported in Table 1. TensorPool achieves 3643 MAC/cycle (FP16) on GEMM: 6× better than TeraPool, thanks to the 2× larger number of FMAs (2.25× including PEs), and 3× better utilization of compute units. Based on power simulation results (Synopsys PrimeTime 2022.03), TensorPool scores 1.53 TFLOPS@16b/W, 106 GFLOPS@16b/W/mm^2: 8.8×, 9.9× higher than TeraPool, thanks to domain specialization.

5.3 SoA Comparison

Table 2 reports a comparison between the current state of the art components for RAN processing. Our clusters stand out, offering the lowest power consumption and the highest ratio between the size of the local data-memory (L1) and the number of PEs with shared low-latency access to it. This feature allows to execute large-dimensional PHY workloads without fragmenting them across many small clusters, thereby adding expensive data movement, as well as distribution and synchronization overheads across memory hierarchies, as it was demonstrated in Section 3.

Table 2: SoA processors for AI-native RAN.

	Node	Frequency [GHz]	Peak.-Perf. [TFLOPS@16b]	Power [W]	L1-size
TensorPool	7nm	0.9	8.3	3	4MiB/(266PEs&16TEs)
TeraPool [3]	12nm	0.88	3.6	5.5	4MiB/1024PEs
ARC Compact [8]	5nm	2.5	455.3	72	128KiB/188PEs
X100 [9]	-	-	-	35	-
Octeon10 [10]	5nm	2.5	-	50	64KiB/PE

In Table 3 we report the main architectural parameters for the TensorPool configuration and the TensorPool-7 configuration, described in [2] as the target configuration for the project. With respect to TensorPool-7, we opted for a more modular configuration, where each Tile of the cluster contains the same number of Snitch cores. We also decided to reduce the number of cores per Tile to 4, to reduce the number of ports in the local interconnect for the RedMulE Tiles. Based on the results on a post placement and routing instance of TensorPool our design matches the requirements set in [2]. In our future work we will continue on the placement and routing of the full cluster to verify consistency of these preliminary results.

6 Future work

According to the discussion in [2], in the next steps of the project:

Table 3: Comparison between the results on the chosen configuration of TensorPool and the values estimated in [2] for the target TensorPool-7 configuration.

Cluster Configuration	TeraPool	TensorPool-7	TensorPool
FLOP/cycle/cluster @16b	2048	4896	4608
NumCores	1024	400	256
NumRedMulETiles	0	16	16
NumSnitchTiles	128	48	48
NumTiles	128	64	64
NumGroups	4	4	4
NumSubGroups	16	16	16
Area-Cluster (TSMC7) [mm^2]	27.81	20.26	14.56
Energy-Efficiency (TSMC7) [GFLOP/s/W]	276.48	1950	1530
Energy/Area-Efficiency (TSMC7) [GFLOP/$s/W/mm^2$]	9.94	65.92	106

1. We will complete the place&route of TensorPool TSMC N7. Previous work on the 2D physical implementation of the TeraPool cluster highlighted area efficiency losses [3]. An area overhead is indeed required to place and route the cores to memory interconnect crossbars within a Group and across different Groups. Moving from this evidence, we will assess the severity of this issue for the TensorPool cluster. To alleviate it, we will propose and evaluate a 3D partition of the cluster that can reduce 2D routing congestion by exploiting the routing resources in the vertical dimension and by increasing the proximity between physical design hierarchies (Groups and SubGroups) through vertical stacking.

2. We will continue on the benchmarking of the basic operators in the NextRAN-AI model described in [2], on the placed&routed TensorPool instance, providing its PPAs.

3. We will evaluate the whole system performance relying on an higher level simulator (e.g. the GVSoC simulator), to speed-up verification and performance analysis at a system level, without the time constraints of RTL-simulation. We will therefore be able to provide results on the end-end system performance.

Acknowledgments

Huawei Technologies Sweden AB supported this work.

References

[1] Yvan Tortorella et al. “RedMule: A mixed-precision matrix–matrix operation engine for flexible and energy-efficient on-chip linear algebra and TinyML training acceleration”. In: *Future Generation Computer Systems* 149 (2023), pp. 122–135. ISSN: 0167-739X.

DOI: https://doi.org/10.1016/j.future.2023.07.002. URL: https://www.sciencedirect.com/science/article/pii/S0167739X23002546.

[2] Marco Bertuletti and Yichao Zhang. *AI-aided Physical Layer Processing*. 2025. URL: https://github.com/pulp-platform/mempool/blob/doc/doc/NextRAN-AI_work/NextRAN_AI_Milestone_Y1.1.pdf.

[3] Yichao Zhang et al. "TeraPool: A Physical Design Aware, 1024 RISC-V Cores Shared-L1-Memory Scaled-up Cluster Design with High Bandwidth Main Memory Link". In: *IEEE Transactions on Computers* (2025), pp. 1–14. DOI: 10.1109/TC.2025.3603692.

[4] Florian Zaruba et al. "Snitch: A Tiny Pseudo Dual-Issue Processor for Area and Energy Efficient Execution of Floating-Point Intensive Workloads". In: *IEEE Transactions on Computers* 70.11 (2021), pp. 1845–1860. DOI: 10.1109/TC.2020.3027900.

[5] Diyou Shen et al. "TCDM Burst Access: Breaking the Bandwidth Barrier in Shared-L1 RVV Clusters Beyond 1000 FPUs". In: *2025 Design, Automation Test in Europe Conference (DATE)*. 2025, pp. 1–7. DOI: 10.23919/DATE64628.2025.10992996.

[6] Marco Bertuletti et al. "A 66-Gb/s/5.5-W RISC-V Many-Core Cluster for 5G+ Software-Defined Radio Uplinks". In: *IEEE Transactions on Very Large Scale Integration (VLSI) Systems* 33.8 (2025), pp. 2225–2238. DOI: 10.1109/TVLSI.2025.3576855.

[7] Lukas Steiner et al. "DRAMSys4.0: An Open-Source Simulation Framework for In-depth DRAM Analyses". In: *Int. J. Parallel Program.* 50.2 (Apr. 2022), pp. 217–242. ISSN: 0885-7458. DOI: 10.1007/s10766-022-00727-4.

[8] *Deploy AI-RAN at Cell Sites with NVIDIA ARC-Compact*. URL: https://developer.nvidia.com/blog/deploy-ai-ran-at-cell-sites-with-nvidia-arc-compact/.

[9] *How we Won the Acceleration Architecture Debate*. URL: https://www.qualcomm.com/news/onq/2023/03/how-we-won-the-acceleration-architecture-debate.

[10] *Data Processing Units, Empowering 5G carrier, enterprise and AI cloud data infrastructure*. URL: https://www.marvell.com/products/data-processing-units.html.